

Avaliação Teórica e Experimental dos Algoritmos de Caminho Mínimo em Matriz: Recursão Pura e Programação Dinâmica

João Vitor Santana Lima e Samuel Torres Vieira da Silva

¹Departamento de Sistemas de Informação – Universidade Federal do Piauí (UFPI)
64.600-000 – Picos – PI – Brasil

{joao.vitor.1, samuel.vieira}@ufpi.edu.br

Resumo. *Este trabalho apresenta uma avaliação teórica e experimental de dois algoritmos para o problema do caminho mínimo em uma matriz, comparando uma solução baseada em recursão pura com uma solução fundamentada em programação dinâmica por tabulação. A investigação combina análise assintótica e experimentação empírica, permitindo relacionar as previsões teóricas com os resultados observados na prática. Os experimentos foram conduzidos utilizando matrizes quadradas de tamanho 8×8 e 13×13 , com 30 execuções por cenário para obtenção de médias representativas. Os resultados evidenciam a diferença expressiva entre as complexidades das duas abordagens: enquanto a solução recursiva apresenta crescimento exponencial e se torna impraticável para matrizes moderadas, a programação dinâmica mantém complexidade polinomial $O(n \times m)$, processando matrizes 13×13 em tempos da ordem de microssegundos. Conclui-se que a programação dinâmica, ao eliminar recomputações por meio da tabulação, transforma um problema computacionalmente custoso em um problema eficientemente solucionável.*

1. Introdução

Problemas de otimização de caminhos constituem uma classe fundamental em ciência da computação, com aplicações que abrangem sistemas de navegação, robótica, jogos digitais e roteamento em redes [Cormen et al. 2009, Sedgewick and Wayne 2011]. Uma instância clássica desse tipo de problema é o caminho mínimo em uma matriz, no qual se busca percorrer uma grade de custos numéricos do canto superior esquerdo ao canto inferior direito, com movimentos restritos às direções para baixo e para a direita, minimizando o custo acumulado ao longo do trajeto [Cormen et al. 2009].

A solução desse problema pode ser obtida por diferentes abordagens algorítmicas, com desempenhos radicalmente distintos. A abordagem recursiva pura, de implementação direta e intuitiva, explora todos os caminhos possíveis sem reaproveitar soluções previamente calculadas para subproblemas repetidos, resultando em complexidade exponencial [Aho et al. 1974]. Em contrapartida, a programação dinâmica, paradigma introduzido por Richard Bellman na década de 1950 [Bellman 1957], calcula cada subproblema exatamente uma vez, armazenando os resultados em uma tabela e eliminando as recomputações desnecessárias.

A análise experimental de algoritmos complementa a análise assintótica formal, permitindo observar diferenças práticas de desempenho que não são capturadas apenas pela notação assintótica [McConnell 2008]. Diante do exposto, o objetivo deste trabalho é

realizar uma avaliação comparativa das abordagens recursiva pura e de programação dinâmica para o problema do caminho mínimo em matriz, analisando desempenho temporal, consumo de memória e complexidade assintótica observada.

2. Referencial Teórico

Nesta seção, são apresentados os fundamentos teóricos das duas abordagens algorítmicas estudadas: a recursão pura e a programação dinâmica por tabulação. São discutidas as características estruturais de cada abordagem, bem como suas complexidades assintóticas de tempo e espaço.

2.1. Abordagem Recursiva

A abordagem recursiva pura traduz diretamente a definição do problema em chamadas recursivas. A partir de uma posição (i, j) , o algoritmo explora recursivamente os caminhos para baixo e para a direita, retornando o mínimo entre os dois custos possíveis somado ao custo da célula atual [Aho et al. 1974]. Os casos base correspondem à chegada na posição final e às posições fora dos limites da matriz.

O problema fundamental dessa abordagem é a ausência de reutilização de resultados: o custo ótimo para cada posição é recalculado tantas vezes quantas ela aparece na árvore de recursão. Isso resulta em uma complexidade de tempo exponencial $O(2^{n+m})$, tornando a abordagem impraticável para matrizes moderadas. A complexidade de espaço é $O(n + m)$, determinada pela profundidade máxima da pilha de chamadas [Sedgewick and Wayne 2011]. A Listagem 1 apresenta o pseudocódigo da abordagem.

Listing 1. Pseudocódigo da Abordagem Recursiva

```
1 FUNCAO caminhoMinRecursivo(matriz, linha, coluna)
2   n_lin = num_linhas(matriz)
3   n_col = num_colunas(matriz)
4
5   // Caso base: canto inferior direito
6   SE linha == n_lin - 1 E coluna == n_col - 1 ENTAO
7     RETORNA matriz[linha][coluna]
8   FIM SE
9
10  // Posicao fora dos limites
11  SE linha >= n_lin OU coluna >= n_col ENTAO
12    RETORNA INFINITO
13  FIM SE
14
15  baixo = caminhoMinRecursivo(matriz, linha + 1, coluna)
16  direita = caminhoMinRecursivo(matriz, linha, coluna + 1)
17
18  RETORNA matriz[linha][coluna] + MIN(baixo, direita)
19 FIM FUNCAO
```

2.2. Programação Dinâmica

A programação dinâmica, formalizada por Richard Bellman em 1957 [Bellman 1957], é um paradigma de projeto de algoritmos que resolve problemas com subestrutura ótima e subproblemas sobrepostos de forma eficiente, calculando cada subproblema exatamente uma vez e armazenando os resultados para reutilização futura [Cormen et al. 2009]. O

princípio subjacente é o **Princípio de Otimalidade de Bellman**: uma solução ótima global é sempre composta por soluções ótimas de seus subproblemas [Bellman 1957].

Aplicada ao caminho mínimo em matriz, a abordagem por **tabulação** (*bottom-up*) preenche iterativamente uma tabela dp de dimensões $n \times m$, onde cada célula $dp[i][j]$ armazena o menor custo acumulado para se chegar à posição (i, j) a partir da origem [Sedgewick and Wayne 2011]. Esse preenchimento garante que cada subproblema seja resolvido uma única vez, resultando em complexidade de tempo $O(n \times m)$ e complexidade de espaço $O(n \times m)$, referente à tabela dp mantida em memória [Cormen et al. 2009]. A Listagem 2 apresenta o pseudocódigo da abordagem.

Listing 2. Pseudocódigo da Programação Dinâmica (Tabulação)

```
1 FUNCAO caminhoMinDinamico(matriz)
2   n_lin = num_linhas(matriz)
3   n_col = num_colunas(matriz)
4
5   CRIA tabela dp[n_lin][n_col]
6   dp[0][0] = matriz[0][0]
7
8   // Preenche primeira coluna (so ha caminho por cima)
9   PARA i = 1 ATE n_lin - 1 FACA
10    dp[i][0] = dp[i-1][0] + matriz[i][0]
11   FIM PARA
12
13   // Preenche primeira linha (so ha caminho pela esquerda)
14   PARA j = 1 ATE n_col - 1 FACA
15    dp[0][j] = dp[0][j-1] + matriz[0][j]
16   FIM PARA
17
18   // Preenche celulas internas combinando os vizinhos
19   PARA i = 1 ATE n_lin - 1 FACA
20    PARA j = 1 ATE n_col - 1 FACA
21    dp[i][j] = matriz[i][j] + MIN(dp[i-1][j], dp[i][j-1])
22    FIM PARA
23   FIM PARA
24
25   RETORNA dp[n_lin - 1][n_col - 1]
26 FIM FUNCAO
```

3. Metodologia

Esta seção descreve o ambiente computacional utilizado nos experimentos, incluindo as especificações de hardware, as estratégias de geração de dados de entrada e os procedimentos adotados para medição e análise de desempenho, visando garantir a reprodutibilidade dos resultados.

3.1. Especificações Técnicas

A Tabela 1 apresenta as especificações da máquina utilizada para a execução dos experimentos.

Tabela 1. Especificações Técnicas da Máquina Utilizada

Especificação	Descrição
Processador	AMD Ryzen 7 5700U
Memória RAM	12,0 GB
Sistema Operacional	Windows 11 Home

3.2. Especificações Práticas

Foram desenvolvidas implementações em linguagem Python para ambos os algoritmos. As matrizes de entrada foram geradas de forma aleatória, utilizando o gerador `Random` da biblioteca padrão do Python com sementes fixas (1001 para o Cenário A e 1002 para o Cenário B), garantindo a reprodutibilidade das entradas. Os valores de cada célula foram amostrados uniformemente no intervalo inteiro $[1, 9]$.

Os experimentos consideraram dois cenários de tamanho de matriz: 8×8 (Cenário A) e 13×13 (Cenário B). Para a medição do tempo de execução, foi utilizada a função `perf_counter` do módulo `time` da biblioteca padrão do Python, que fornece um contador de alta resolução e comportamento monotônico, adequado para medições de curta duração [McConnell 2008]. Cada cenário foi executado 30 vezes por algoritmo, sem descarte de execuções iniciais, e a média aritmética dos tempos foi utilizada como representante estatístico do desempenho.

A estimativa de memória foi realizada de forma analítica: para a abordagem recursiva, estimou-se o produto entre a profundidade máxima da pilha de chamadas ($2n - 1$) e o tamanho de um frame de execução obtido via `sys.getsizeof`; para a programação dinâmica, somou-se o custo estrutural das listas Python utilizadas como tabela dp e o custo dos dados inteiros armazenados. Os resultados foram consolidados em arquivos CSV e os gráficos foram gerados com a biblioteca `matplotlib`. A Tabela 2 resume os critérios de avaliação adotados.

Tabela 2. Critérios de Avaliação e Justificativas

Critério de Avaliação	Justificativa para a Escolha
Tempo Médio de Execução (s)	Permite avaliar o desempenho temporal dos algoritmos, sendo a métrica mais direta para comparar a eficiência prática em diferentes tamanhos de entrada.
Estimativa de Memória (bytes)	Permite comparar o custo de espaço das duas abordagens, contrastando o uso da pilha de chamadas na recursão com a tabela de tabulação na programação dinâmica.
Complexidade Assintótica Observada	Permite verificar empiricamente a taxa de crescimento do custo computacional em função do tamanho da entrada, possibilitando a comparação entre o comportamento teórico e o prático.

4. Resultados e Discussão

Nesta seção, são apresentados e analisados os dados obtidos nos experimentos de desempenho para os dois algoritmos avaliados. Os resultados são discutidos à luz das análises

teóricas apresentadas na Seção 2, buscando relacionar os resultados empíricos com o comportamento esperado de cada abordagem.

4.1. Tempo de Execução

A Tabela 3 consolida os tempos médios de execução obtidos em 30 experimentos para cada combinação de algoritmo e cenário, junto ao custo mínimo calculado em cada caso.

Tabela 3. Tempos Médios de Execução e Resultados (30 execuções)

Cenário	Algoritmo	Tempo Médio (s)	Custo Mínimo
A (8×8)	Recursivo	$3,52 \times 10^{-3}$	29
A (8×8)	Prog. Dinâmica	$9,89 \times 10^{-6}$	29
B (13×13)	Recursivo	3,07	74
B (13×13)	Prog. Dinâmica	$2,37 \times 10^{-5}$	74

A análise do Cenário A (Figuras 1 e 2) revela que, mesmo para uma matriz de apenas 8×8, a diferença de desempenho entre os dois algoritmos já é expressiva. A programação dinâmica executa em $9,89 \times 10^{-6}$ s, enquanto a abordagem recursiva demanda $3,52 \times 10^{-3}$ s, uma aceleração de aproximadamente **356 vezes** em favor da programação dinâmica.

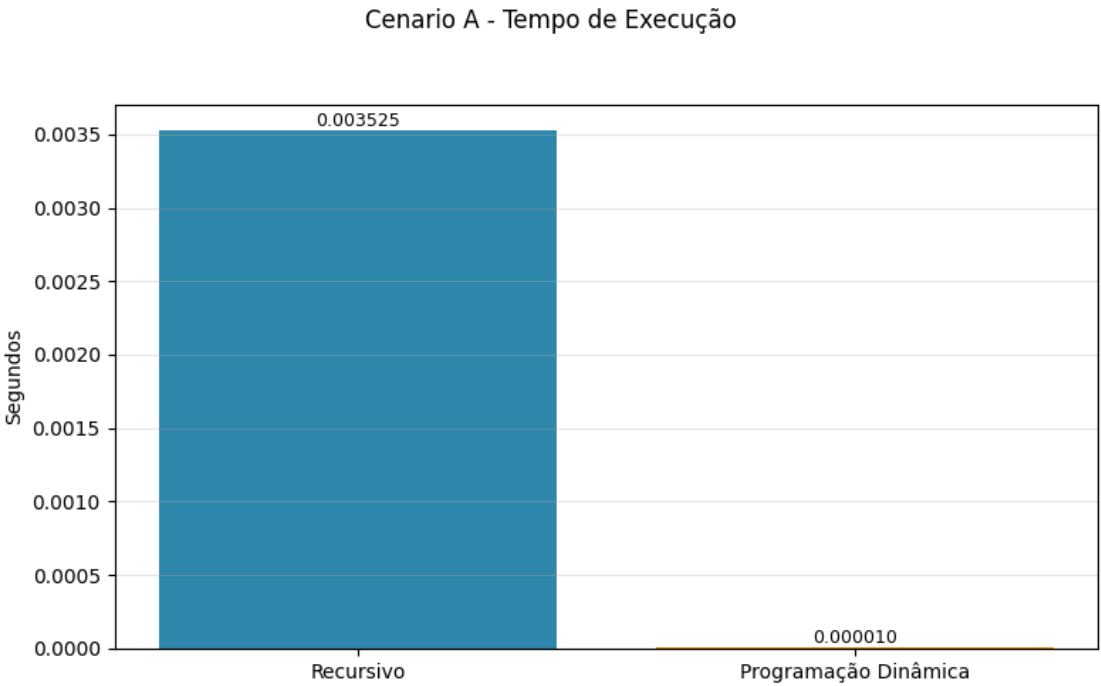


Figura 1. Tempo Médio de Execução, Cenário A (8×8)

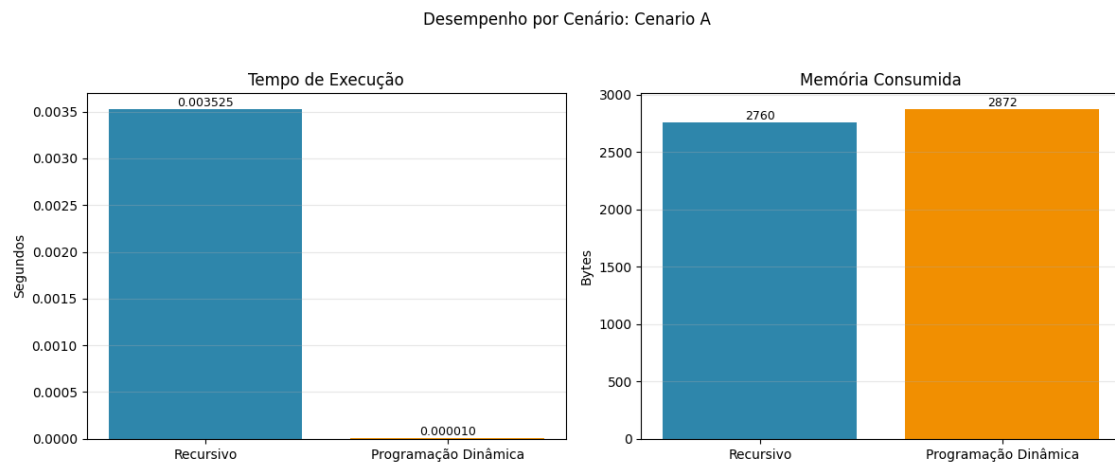


Figura 2. Comparativo de Desempenho, Cenário A (8×8)

O Cenário B (Figuras 3 e 4) expõe de forma expressiva o impacto da complexidade exponencial da recursão pura. Para a matriz 13×13 , o tempo médio da abordagem recursiva foi de 3,07 s, enquanto a programação dinâmica concluiu a mesma tarefa em $2,37 \times 10^{-5}$ s, uma diferença de aproximadamente **129.500 vezes**. A validação cruzada dos resultados, onde ambos os algoritmos retornaram o mesmo custo ótimo (29 no Cenário A e 74 no Cenário B), confirma a corretude das implementações.

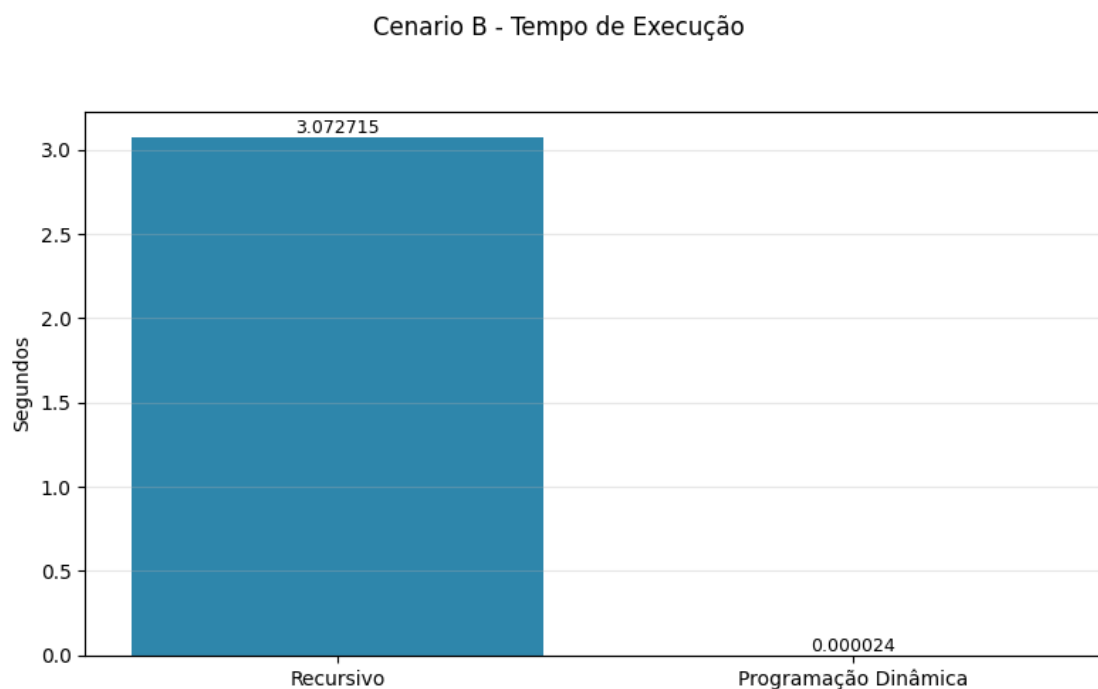


Figura 3. Tempo Médio de Execução, Cenário B (13×13)

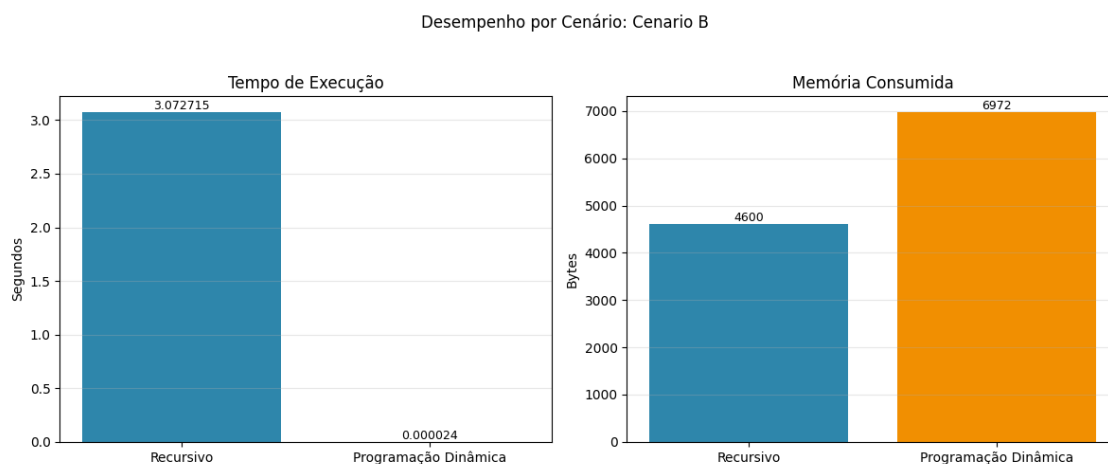


Figura 4. Comparativo de Desempenho, Cenário B (13×13)

4.2. Consumo de Memória

A Tabela 4 apresenta as estimativas de consumo de memória para cada combinação de algoritmo e cenário. As Figuras 5 e 6 ilustram os valores de forma visual.

Tabela 4. Estimativas de Consumo de Memória

Cenário	Algoritmo	Memória Estimada (bytes)
A (8×8)	Recursivo	2.760
A (8×8)	Prog. Dinâmica	2.872
B (13×13)	Recursivo	4.600
B (13×13)	Prog. Dinâmica	6.972

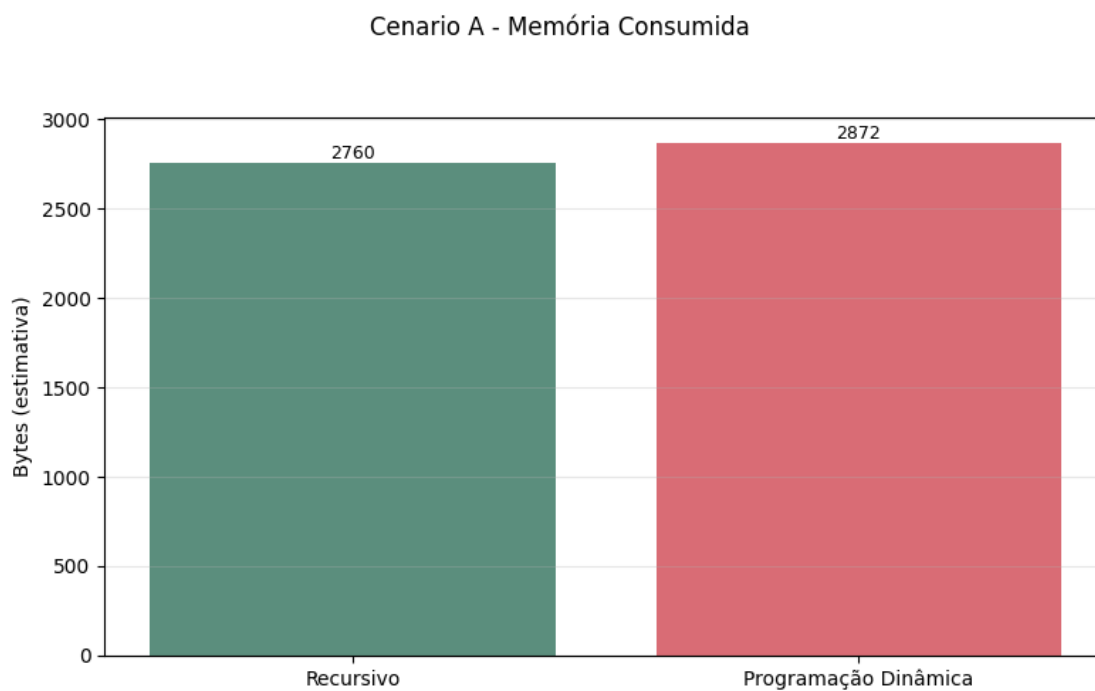


Figura 5. Estimativa de Memória Consumida, Cenário A (8×8)

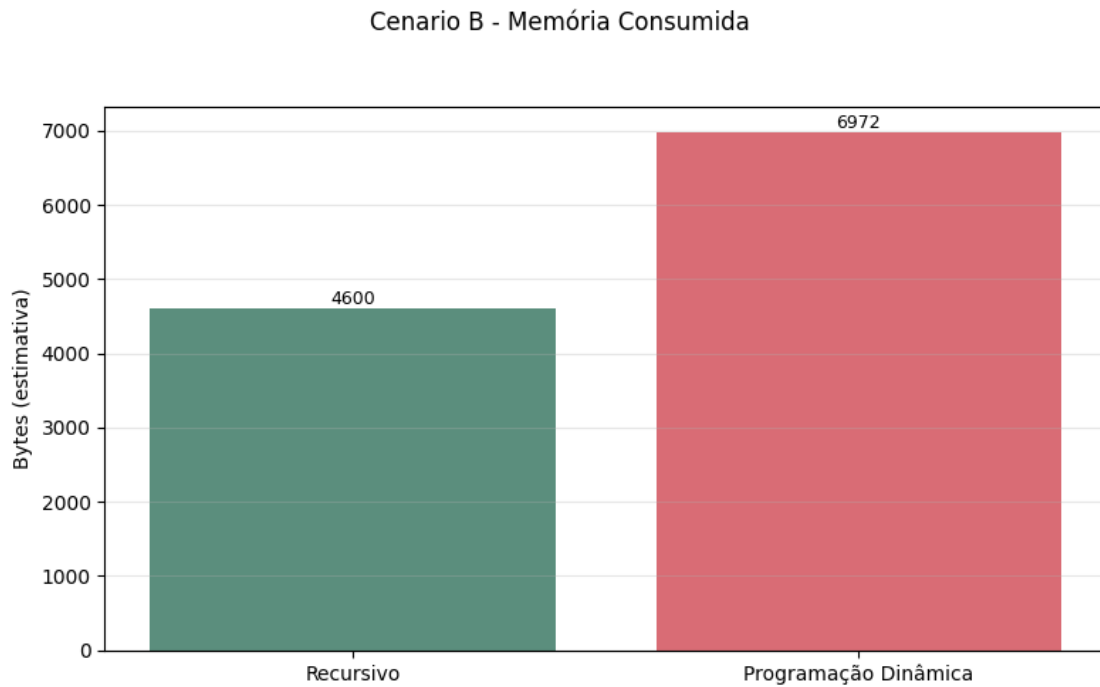


Figura 6. Estimativa de Memória Consumida, Cenário B (13×13)

Os resultados revelam que a abordagem recursiva apresenta consumo de memória inferior em ambos os cenários. Isso se explica pelas complexidades de espaço distintas das duas abordagens: a recursão pura utiliza memória proporcional apenas à profundidade máxima da pilha de chamadas, $O(n + m)$, enquanto a programação dinâmica mantém a tabela dp inteiramente em memória ao longo de toda a execução, com complexidade $O(n \times m)$ [Cormen et al. 2009, Sedgewick and Wayne 2011]. Para os tamanhos avaliados, essa diferença ainda é pequena em valores absolutos, mas a tendência se acentua com o crescimento da matriz.

4.3. Complexidade Assintótica Observada

A análise empírica da taxa de crescimento entre os cenários A e B permite verificar a aderência dos resultados às complexidades teóricas de cada abordagem, conforme resumidas na Tabela 5.

Tabela 5. Complexidade Assintótica dos Algoritmos

Abordagem	Tempo	Espaço
Recursiva	$O(2^{n+m})$	$O(n + m)$
Dinâmica	$O(n \times m)$	$O(n \times m)$

Programação Dinâmica ($O(n \times m)$): A razão entre os tempos médios do Cenário B e do Cenário A foi de $2,37 \times 10^{-5} / 9,89 \times 10^{-6} \approx 2,40$. A previsão teórica para esse crescimento, com base na complexidade $O(n \times m)$, é de $13^2 / 8^2 \approx 2,64$. A aderência é satisfatória, com discrepância de aproximadamente 9% atribuída ao *overhead* do interpretador Python [Weiss 2014].

Recursão Pura ($O(2^{n+m})$): A razão entre os tempos médios do Cenário B e do Cenário A foi de $3,07 / 3,52 \times 10^{-3} \approx 872$. A previsão teórica, com base na complexidade $O(2^{n+m})$, é de $2^{26} / 2^{16} = 2^{10} = 1.024$. A razão observada apresenta desvio

de aproximadamente 15% em relação à previsão teórica, consistente com variações nas constantes de proporcionalidade e no *overhead* do interpretador [Knuth 1997]. Em ambos os casos, o crescimento é claramente de natureza exponencial, confirmando a inadequação da abordagem recursiva pura para matrizes de tamanho moderado.

5. Conclusão

Os resultados experimentais confirmaram a superioridade da programação dinâmica sobre a recursão pura para o problema do caminho mínimo em matriz. No Cenário A (8×8), a programação dinâmica foi aproximadamente 356 vezes mais rápida; no Cenário B (13×13), essa vantagem escalou para cerca de 129.500 vezes. Esse comportamento é consistente com as complexidades teóricas: $O(n \times m)$ para a tabulação e crescimento exponencial $O(2^{n+m})$ para a recursão pura.

Em relação ao consumo de memória, a abordagem recursiva utiliza menos espaço nos cenários avaliados, pois consome memória proporcional apenas à profundidade da pilha de chamadas, $O(n + m)$, enquanto a tabulação requer a alocação de uma tabela completa $O(n \times m)$. Contudo, essa vantagem de espaço não compensa o custo temporal da recursão sem memoização.

A concordância entre os fatores de crescimento empíricos e os previstos teoricamente valida tanto as implementações quanto os modelos analíticos adotados. Conclui-se que a programação dinâmica, ao aplicar o Princípio de Otimalidade de Bellman e eliminar recomputações por tabulação, transforma um problema de complexidade exponencial em um problema eficientemente solucionável, reforçando sua relevância como paradigma central no projeto e análise de algoritmos [Bellman 1957, Cormen et al. 2009].

Esses resultados reforçam a importância de avaliar algoritmos não apenas do ponto de vista teórico, mas também experimental, prática amplamente defendida na literatura clássica de análise de desempenho de software e algoritmos [Weiss 2014].

Referências

- Aho, A. V., Hopcroft, J. E., and Ullman, J. D. (1974). *The Design and Analysis of Computer Algorithms*. Addison-Wesley.
- Bellman, R. (1957). *Dynamic Programming*. Princeton University Press, Princeton, NJ.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). *Introduction to Algorithms*. MIT Press, Cambridge, MA, 3 edition.
- Knuth, D. E. (1997). *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. Addison-Wesley, Reading, MA, 3 edition.
- McConnell, S. (2008). *Code Complete*. Microsoft Press, 2 edition.
- Sedgewick, R. and Wayne, K. (2011). *Algorithms*. Addison-Wesley Professional, Upper Saddle River, NJ, 4 edition.
- Weiss, M. A. (2014). *Data Structures and Algorithm Analysis in Java*. Pearson, 3 edition.